

AI651 Deep Learning for Space, Time and Graphs

Programming Assignment 1

Ahmed Jawed

October 2026

Code repository: <https://github.com/ahmedjawedaj/DL4STG-PA1>

Contents. Task 1 (implementation, five responses, figures from the PA1_PRESET=full run). Task 2 (Autoformer pipeline, validation, required optional-data ablation, submission). Generative-AI disclosure at the end.

1 Task 1: Forecasting Across Heterogeneous Sensors

Implementation summary

- **RawAttentionForecaster.forward:** circular `Conv1d` embedding of the $[B, 1, L]$ context, add the learned position table, two encoder blocks, final `LayerNorm`, flatten $[B, L \cdot d]$ and a linear head to $[B, H]$.
- **SeriesDecomposition.forward:** replicate the first and last sample $\lfloor m/2 \rfloor$ times, `avg_pool1d` with stride 1 along time, return $(x - T, T)$ so that $S + T = x$ exactly.
- **aggregate_delays:** build the source index $(t - \tau_j) \bmod L$ for every example and selected delay, **gather** the shifted values, weight and sum over j . Fully vectorised and differentiable in values and weights.

All notebook checks pass, including the worked decomposition and delay-aggregation examples.

Response 1A: expected versus measured ordering (Output 1.1)

Expected ordering, written before Output 1.1. Established stations: period-routed ridge < Raw Attention < sensor-specific ridge $\approx$ shared ridge (RMSE). Held-out station 4: the same, with sensor-specific ridge unavailable and every model slightly worse than on stations 1–3.

Measured (validation, RMSE in m/s²).

Model	Established	Held-out 4	Params	Fit (s)
Shared ridge	0.767	0.876	4 608	0.01
Sensor-specific ridge	0.768	n/a	13 824	0.09
Period-routed ridge	0.166	0.173	59 904	0.03
Raw Attention	0.426	0.607	368 368	928

- **Ordering held:** routed ridge < Raw Attention < shared $\approx$ sensor-specific on both populations and in every product.
- **Size of the gaps:** routing cuts shared-ridge RMSE by 78% on stations 1–3 (0.767 to 0.166). Raw Attention recovers only about 57% of that gain (0.426) with $6\times$ the parameters of the routed ridge.
- **Sensor identity adds nothing:** sensor-specific ridge is 0.768 against 0.767 for shared. Per-product RMSEs match to within 0.01 (A 0.804 vs 0.805, C 0.630 vs 0.621). Station identity is not the variable that changes the rule.
- **Across products:** shared ridge is worst on Product B (0.854) and best on C (0.621). Routed ridge is flat across products (0.181, 0.164, 0.152).
- **Held-out station:** routed ridge barely moves (0.166 to 0.173, +4%). Raw Attention degrades by 42% (0.426 to 0.607). A plausible (untested) explanation is that its learned mixing partly fits the amplitude, mounting phase and drift of the three fitted stations.
- **Revision:** I expected Raw Attention to be closer to the routed ridge. The explicit pace variable is worth more than generic context-dependent mixing in this world, and it transfers to an unseen sensor because pace is shared by the whole line while sensor details are not.

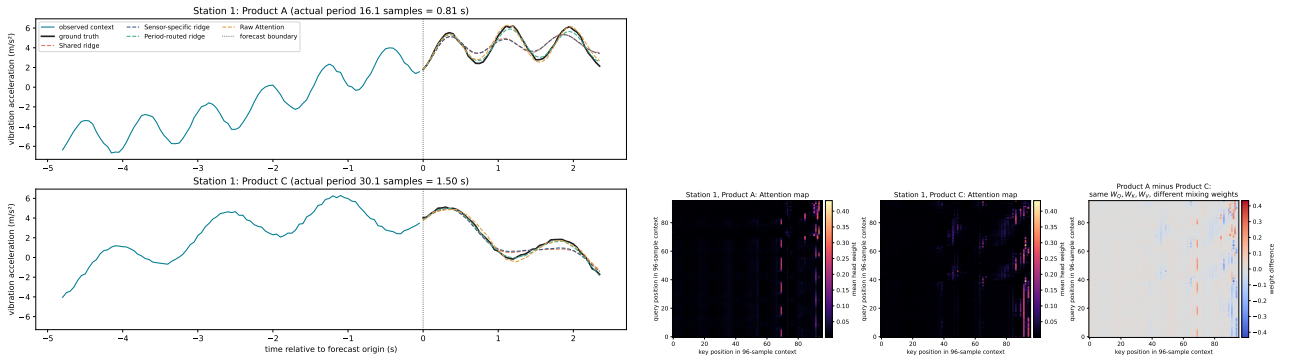


Figure 1: Output 1.2 (left): station 1 under Products A and C. Output 1.3 (right): Attention maps for the two contexts and their difference.

Response 1B: why the rule must depend on context (Outputs 1.2, 1.3)

- **The compromise in Output 1.2 (validation windows).** On station 1, Product A (period 16.1), shared and sensor-specific ridge produce a damped oscillation: amplitude is roughly halved and the peaks drift slightly out of phase with the truth (window RMSE 0.779 and 0.787). On Product C (period 30.1) the same matrix fits much better (0.448 and 0.436).
- **Why.** $\widehat{W}$ in Eq. (8) is one fixed linear map. Continuing a cycle of period 16 and one of period 30 requires different phase advances from the same lags. Least squares averages over all training paces. Continuations that disagree in phase partly cancel, so the forecast shrinks in amplitude and its phase is only approximately right. The damping is the visible cost of averaging incompatible continuations.
- **Routed ridge avoids it** because each pace band has its own $\widehat{W}$ (0.232 and 0.150 on the same windows). Raw Attention also tracks both (0.287 and 0.270) without being told the pace.
- **How Attention changes the operator (Output 1.3, validation windows).** W_Q, W_K, W_V are the same for both windows. The weights $\text{softmax}(QK^\top/\sqrt{d_h})$ are recomputed from each window, so the mixing matrix differs: mean absolute difference 0.011 per entry but up to 0.433 on individual query and key pairs. The Product A map concentrates on a few key columns late in the context, the Product C map spreads over bands near positions 40 to 95.
- **Contrast.** Ridge applies $\hat{z} = x^\top \widehat{W}$ with the same $\widehat{W}$ for every context. Attention applies $\hat{z} = f(A(x)V(x))$ where $A(x)$ is built from the input. Output 1.3 shows only that the mixing depends on the context, not that it has recovered the physical period.

Response 2: separating slow structure from repetition (Outputs 2.1–2.3)

- **Selected width:** $m = 41$, chosen by the harness on training windows of stations 1–3 via the oracle recovery score (Output 2.2: raw 0.687, widths 17/25/33/41 give 0.980, 0.989, 0.993, 0.998).
- **Visual change (Output 2.1, validation window, station 2, Product C, period 32).** Width 17 leaves the trend bending with each vibration cycle and the residual has uneven peaks. As m grows the trend becomes a smooth S-shaped ramp and the residual becomes a clean, centred oscillation (residual std grows from 0.58 to 1.57 m/s² because less of the cycle is absorbed into T). The period projection score has a clear (broad) peak near 32 after decomposition and is nearly flat on the raw context. Edge windows are slightly biased because of endpoint replication.
- **Empirical change (Output 2.3, validation).** Attention + decomposition vs Raw Attention: established RMSE 0.426 to 0.297 (−30%, MSE 0.182 to 0.088), held-out 0.607 to 0.441 (−27%). Every product improves on both populations, for example held-out Product A 0.616 to 0.368. Parameters rise only from 368368 to 373024.
- **Why a moving average fits this world.** The drift has a 240-sample scale, 6–17 times longer than the 14–36-sample vibration. A centred box filter of width m removes a sinusoid of period P exactly when m is a multiple of P and attenuates it strongly when $m > P$, while passing a slow ramp almost unchanged. The 41-sample window is longer than every pace yet much shorter than the drift scale, so it separates the two by time scale.
- **Why the oracle diagnostic is a sound selector.** It uses training windows only and a quantity (the true period) that exists only in simulation to score whether S exposes the repetition. It is not an input to any forecaster, so it selects the filter without touching validation or test data.
- **Where it fails / alternative.** If the drift contained a step change or a curvature on the same time scale as the vibration (e.g. a load jump mid-context), the box filter would smear the step into both streams. STL or

a local-polynomial (LOESS) trend assumes a smooth but locally curved trend and fits edges with one-sided polynomials, and a learnable (trainable-kernel) decomposition assumes the separation itself should be learned from data.

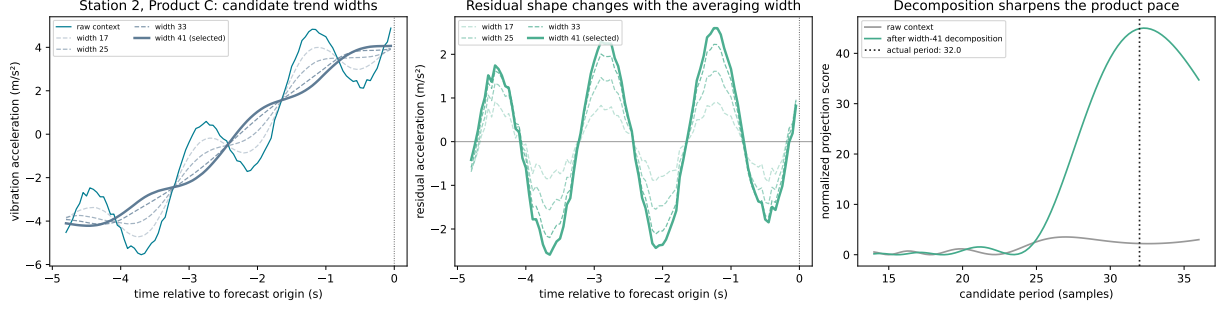


Figure 2: Output 2.1: candidate trend widths, residuals and the period projection before and after width-41 decomposition.

Response 3: mixing by temporal delay (Outputs 3.1, 3.2)

- **Checks.** Output 3.1 reproduces the direct circular correlation $[4, -4, 4, -4]$ for $[1, 3, 1, 3]$. The worked aggregation gives $z_0 = \frac{3}{4}v_3 + \frac{1}{4}v_1 = 35$, confirming position t reads $t - \tau$.
- **Strongest recurrence (Output 3.2, validation window).** For station 2, Product A (actual period $P = 16.13$), the residual correlation peaks at lags 16 and 32, i.e. P and $2P$ rounded to integers, with correlations 0.896 and 0.897, and again near 48 ($3P$, about 0.89). The troughs near 8, 24 and 40 are the half-period anti-alignments. Integer multiples appear because a shift by kP also aligns peaks with peaks.
- **Why decomposition cleans the evidence.** On the raw context the curve falls from about 0.45 at lag 8 to about -0.55 near lag 40, with only a small plateau around lags 26 to 32 and no peak at 16 or 32. A rising ramp stays similar to itself after small shifts and dissimilar after large circular shifts (the wrap puts the high end next to the low end), so the drift dominates every lag. Removing T leaves the oscillation, and lags that are multiples of P then stand out by about 0.9. The delay mixer ranks delays by exactly this kind of score (on learned q, k), so it can only choose good delays if the slow part does not swamp the ranking.
- **Consequence in the models (Output 4.1).** The delay mixer alone gives the largest single gain (established RMSE 0.426 to 0.219) and delay + decomposition is best among neural models (0.199), consistent with this diagnostic.
- **Where the bias is wrong.** An isolated transient, such as a bearing knock or a jam that appears once in the context, has no recurrence. Circular delay scores spread its energy over many lags, top- K selection then copies shifted versions of the transient into positions where it never happened, and the forecast either echoes a spurious repeat or averages it away. The same happens when the period drifts within the context, because no single delay aligns the early and late cycles.

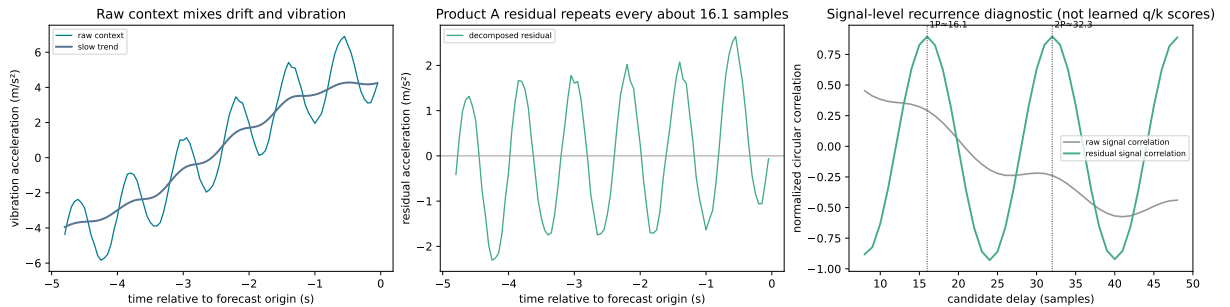


Figure 3: Output 3.2: signal-level recurrence diagnostic (raw vs decomposed residual correlation), not learned q/k scores.

Response 4: compare designs and deploy (Outputs 4.1–4.3)

(a) **Prediction, written before Output 4.1.** Decomposition should help most when the slow component is prominent, because it removes the ramp before the mixer sees it and gives the trend its own linear head. The delay mixer on raw input would rank delays by the ramp’s self-similarity instead of the vibration.

(b) Validation errors, established stations (MSE in $(\text{m/s}^2)^2$).

Model	RMSE	MSE	Held-out RMSE	Held-out MSE	Params
Shared ridge	0.767	0.588	0.876	0.767	4 608
Sensor-specific ridge	0.768	0.590	n/a	n/a	13 824
Period-routed ridge	0.166	0.028	0.173	0.030	59 904
Raw Attention	0.426	0.182	0.607	0.369	368 368
Raw delay mixer	0.219	0.048	0.342	0.117	368 370
Attention + decomposition	0.297	0.088	0.441	0.195	373 024
Autoformer-inspired	0.199	0.039	0.239	0.057	373 026

Interpretation. Both switches help on their own. On stations 1–3 (validation MSE), decomposition lowers Raw Attention by 0.094 and delay mixing by 0.134. The combined model gains 0.142, less than their sum (0.227), so the gains overlap rather than stack. This is consistent with both switches exploiting the same drift-plus-cycle structure, although the 2×2 design cannot identify the cause. Station 4 shows the same pattern (gains 0.174 and 0.252 alone, 0.311 combined). The switches are not uniformly additive per window either: in Output 4.1’s contrasting cases the combined model cuts a station-4 Product B window from 1.386 to 0.189 but loses on a station-2 Product A window (0.353 vs 0.139 for the raw delay mixer). Along the horizon (Output 4.2, validation, stations 1–3), every model starts near 0.11–0.15 m/s^2 at step 1 and diverges afterwards. Raw Attention grows $3.5\times$ by step 48 (0.151 to 0.525), the Autoformer-inspired model only $1.7\times$ (0.136 to 0.235). On station 4 the raw delay mixer degrades sharply late in the horizon (0.635 at step 48), whereas the combined model stays near 0.30 (station 4).

(c) **Deployment.** Period-routed ridge for **both** populations. It has the lowest validation RMSE on stations 1–3 (0.166 vs 0.199 for the best neural model) and on station 4 (0.173 vs 0.239), about $6\times$ fewer parameters and a closed-form fit in under 0.1 s instead of 1424 s. It needs no station identity, so it transfers to station 4. Test results (Output 4.3) confirm the choice without being used for it: 0.161 on stations 1–3 and 0.183 on station 4.

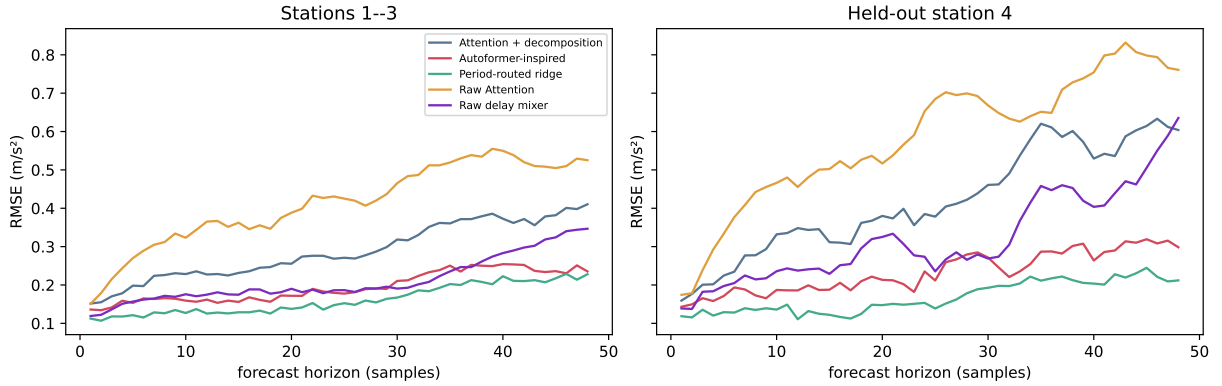


Figure 4: Output 4.2: validation RMSE along the 48-step horizon, stations 1–3 (left) and station 4 (right).

2 Task 2: Leaderboard Challenge

2.1 Data and what it implies

- **Period recovery.** The spectrum and autocorrelation of the training target show a clear 24-step cycle (mean profile peaks at phase 0–1, trough at 14–16). The rest is broad low-frequency power with no single cycle dominating. Autocorrelation is 0.97 at lag 1, 0.40 at lag 24, 0.16 at 48 and about 0.02 by lag 96, so most of the 168-step horizon cannot be solved by short-lag persistence alone.
- **Heavy tail.** Median 73, 99th percentile 418, maximum 994. The level changes by an order of magnitude within days (Figure 6, left).
- **Optional file.** Six continuous variables (A, B, C look like smooth weather states, D, E, F are non-negative cumulative counters) and four one-hot indicators. The strongest contemporaneous links to the target are D (-0.35 with $\log(1 + y)$), H (-0.35), A ($+0.31$) and I ($+0.23$). The file covers the hidden 168 steps, so it can enter as a *known-in-advance* input.

2.2 Validation protocol

- Chronological three-way split into roughly year-long blocks (assuming the 24-step cycle is one day, so a year is about 8 760 steps): fit on $[0, 26304)$, early-stop on $[26304, 35064)$ (origins every 24 steps), compare designs on the final block, whose last 8 568 steps $[35088, 43656)$ are tiled by **51 non-overlapping 168-step blocks**.
- Every block starts at phase 0, exactly like the hidden block (43 656 is a multiple of 24), so per-step errors are comparable with the test week.
- Every configuration is trained with **3 seeds**. I report the seed mean and standard deviation, plus a paired per-block comparison (seed-averaged block RMSE, exact two-sided sign test over the 51 blocks). Seed standard deviation (0.6 to 4.2 RMSE, 10.4 for the log target) is as large as most design differences, so single-seed rankings were not used.
- I also tracked the 16 blocks nearest the start and end of the final block, because the hidden week has the same phase in the recovered 24-step cycle and may be closer to those edge blocks than to the middle of the year.
- The leaderboard was not used for any decision.

2.3 Model

Autoformer encoder-decoder written from scratch (`pa1q2/autoformer.py`), following Wu et al. (2021) and the structure of the public reference `thuml/Autoformer`, reusing the delay scoring and aggregation convention from Question 1.

- **Series decomposition:** centred moving average (kernel 25, replicate padding) at the input and after every sublayer (2 per encoder layer, 3 per decoder layer). The decoder accumulates the three extracted trends through a projection into the trend path, which starts from the input trend (label part) and the window mean (horizon part).
- **Auto-Correlation:** FFT delay scores $R(\tau)$ from learned q, k , top- k delays with $k = \lfloor \ln L \rfloor$ ($k = 5$ in the encoder, 5 in the decoder), softmax weights and circular time-delay aggregation of v . It replaces dot-product attention in encoder self, decoder self and decoder cross mixing.
- Sizes: $L = 168$ (one week), label length 48, `pred_len` = 168, one encoder and one decoder layer, $d_{\text{model}} = 16$, 4 heads, $d_{\text{ff}} = 32$, dropout 0.1.
- **Inputs:** the target (globally standardised on the fit region, no log) as the value channel. Known-in-advance “marks” over the encoder window and the whole decoder window: index-derived phase features ($\sin/\cos$ with periods 24 and 8 766), the ten optional variables (D, E, F through $\log(1 + x)$, all standardised on the fit region), their trailing 6- and 24-step means, and two origin-relative position features (steps since origin/168 and $e^{-h/24}$).
- Training: MSE on the standardised target, Adam, learning rate $3 \cdot 10^{-4}$ halved every epoch (reference schedule), batch 64, gradient clip 1, at most 6 epochs, early stopping with patience 2 on raw-scale RMSE. One epoch is a full pass over every stride-1 window of the fit region.

2.4 Optional-data ablation (required)

Optional data	Params	ES RMSE	Holdout RMSE	MAE	sMAPE	Winter RMSE
none	21 569	102.6	97.7 ± 2.8	72.2	74.9	119.4
past only (encoder values)	27 329	104.9	99.0 ± 0.9	72.2	78.6	119.9
past + horizon (marks)	23 489	79.2	74.4 ± 1.3	51.7	72.8	88.2

Same architecture ($d_{\text{model}} = 32$ base), 3 seeds each, holdout = 51 blocks of the final year. Reference baselines on the same blocks: climatology 93.0, mean of the last week 97.9, persistence 147.7, direct ridge without covariates 87.7, direct ridge with known-horizon covariates 71.5 (about 311k weights).

- **Known-horizon covariates are the main source of skill:** RMSE falls from 97.7 to 74.4 (−24%), MAE from 72.2 to 51.7, winter RMSE from 119.4 to 88.2. Like for like ($d=32$), the covariate model beats the no-data model on 44 of 51 blocks (sign test $p \approx 10^{-7}$). The selected small configuration beats it on 48 of 51.
- **Past-only covariates do nothing** (99.0 vs 97.7, within seed noise). At a 168-step horizon, the conditions that drive the target *during* the week matter, not the ones before the origin. This is the answer to “which portions” and “where they enter”: the horizon part must reach the decoder as a known input.
- **Without optional data the Autoformer is worse than climatology** (97.7 vs 93.0). Its horizon trend starts at the window mean, which for this series is a noisy estimate of the next week’s level (the last-week mean alone scores 97.9).
- Trailing 6/24-step covariate means help a little. Removing them raises mean block RMSE by 1.1 and is worse on 34 of 51 blocks, but the pooled RMSE is almost unchanged.

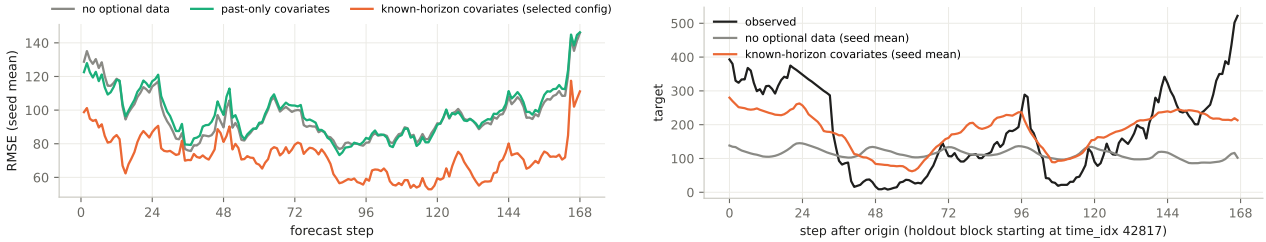


Figure 5: Left: per-step holdout RMSE (seed mean). The known-horizon model is better at every step. The wave follows the 24-step cycle in variance because all blocks start at phase 0. Right: the most volatile late-year holdout week.

2.5 Other design decisions (all 3 seeds, same protocol)

Variant (all with known-horizon covariates)	Params	ES	Holdout	Winter	Epochs
$d=32$ base	23 489	79.2	74.4 ± 1.3	88.2	5.3
log(1 + y) target	23 489	84.4	81.6 ± 10.4	88.2	4.7
no trailing covariate means	22 209	81.6	74.3 ± 0.6	88.6	5.7
width-5 covariate embedding	32 193	79.7	74.0 ± 3.2	84.5	4.0
$d=16$	6 625	80.8	75.1 ± 1.1	87.1	5.0
$d=16$, width-5 embedding	10 977	79.1	74.5 ± 2.6	85.5	3.7
+ last-value anchoring	10 977	78.2	75.7 ± 4.0	84.6	5.0
+ horizon marks + anchoring (selected)	11 297	77.6	72.9 ± 1.9	85.4	3.7
+ horizon marks	11 297	77.5	73.2 ± 2.4	82.9	3.3

The two horizon-mark rows also use replicate instead of circular padding in the value embedding and decoder trend projection, which removed an error spike at the last horizon steps caused by wrap-around (step-168 RMSE 122 to 113 in the anchored pair).

- **Raw vs log target.** Training on $\log(1 + y)$ gives the best MAE (49.4) and sMAPE (54.2) of any run but worse and unstable RMSE (81.6 ± 10.4). The back-transform under-predicts the heavy peaks, and RMSE punishes those misses more than MAE or sMAPE. Since ranking is by RMSE, I train on the raw scale.
- **Size.** Halving the width (23k to 7k or 11k parameters) costs nothing measurable. All covariate variants are within about 3 RMSE, inside seed spread, so the cheaper model is preferred under the $\alpha P + \beta E$ penalty.
- **Horizon position.** Autoformer has no positional encoding, so its decoder cannot tell step 1 from step 100. At step 1 the $d=16$ width-5 model’s RMSE was 107 against 25 for persistence. Adding origin-relative marks

lowered ES RMSE (79.1 to 77.5) and holdout RMSE (74.5 to 73.2) at +320 parameters, and gave the best winter RMSE. Anchoring on the last value was not a clear single-seed win once the marks were present, but its seed-averaged holdout RMSE was slightly better. Starting the horizon trend at the last value failed badly in quick checks, probably because the smoothed decoder trend could not pull a spike back down.

- **Selection rule.** Among variants within noise, I mainly used full holdout RMSE, with winter RMSE and cost as tie-breakers. The two horizon-mark rows are very close (ES 77.5 vs 77.6, holdout 73.2 vs 72.9, paired block difference 0.15 with 25 of 51 blocks each way). I kept the anchored row for the final forecast because its three-seed average had the best full-holdout RMSE, while preserving the non-anchored two-seed forecast as a fallback snapshot.

2.6 Final model and submission

- I refit the selected configuration on $[0, 35064)$ with the 51 holdout blocks as the early-stopping set. Based on the three-seed dev runs, I averaged seeds 0, 1 and 2 after applying the same non-negativity floor. On the final-year early-stop blocks, the best checkpoints reached RMSE 68.3, 79.0 and 68.0 respectively. Seed 1 was weak by itself, but it improved the same ensemble on the dev holdout.
- Forecast origin 43 656 (0-based), i.e. `time_idx` 43 657 to 43 824, in chronological order. Outputs below the 1st percentile of the fit-region target (7.0) are raised to it before averaging. On the dev holdout, the same three-seed anchored ensemble scored RMSE 69.34. I also saved the previous non-anchored seed-0/seed-2 forecast separately before replacing the active submission folder.
- **Declared $P = 33891$ trainable parameters and $E = 15$ epochs.** For an ensemble the leaderboard asks for the sum over the three runs: $11\,297 \times 3$ parameters and $6 + 3 + 6$ epochs. The commands below reproduce the three runs; the final prediction file is the average of their floored forecasts.

```
python -m palq2.train --cov future --seed 0 --d-model 16 --d-ff 32 \
--mark-kernel 5 --horizon-mark 1 --anchor 1 --pad replicate \
--stage final --out runs/final --tag final_anchor_rep
python -m palq2.train --cov future --seed 1 --d-model 16 --d-ff 32 \
--mark-kernel 5 --horizon-mark 1 --anchor 1 --pad replicate \
--stage final --out runs/final --tag final_anchor_rep
python -m palq2.train --cov future --seed 2 --d-model 16 --d-ff 32 \
--mark-kernel 5 --horizon-mark 1 --anchor 1 --pad replicate \
--stage final --out runs/final --tag final_anchor_rep
python -m palq2.submission runs/final/future-log0-ph1-L168-d16-s0-final_anchor_rep.json \
runs/final/future-log0-ph1-L168-d16-s1-final_anchor_rep.json \
runs/final/future-log0-ph1-L168-d16-s2-final_anchor_rep.json
```

- Sanity check only: the active forecast has 168 values in order from `time_idx` 43 657 to 43 824, with minimum 7.0 after flooring.

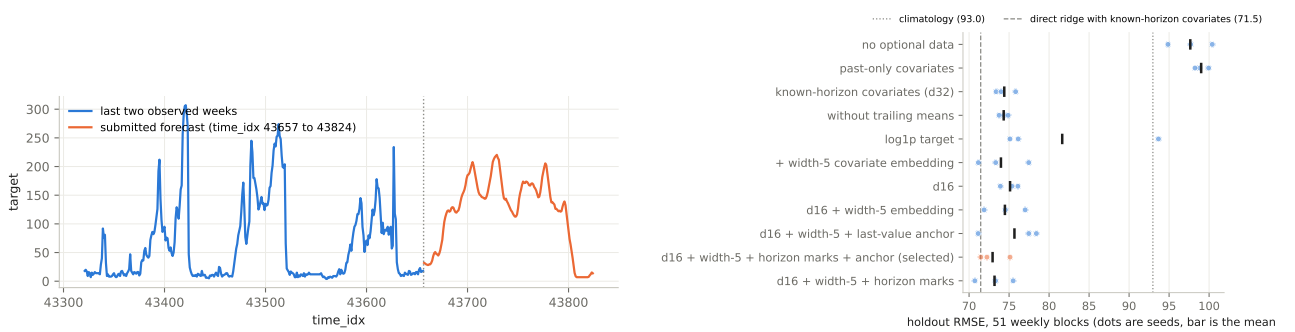


Figure 6: Left: last two observed weeks and the submitted forecast. Right: every run in the ablation (holdout RMSE, 51 blocks). Dashed and dotted lines are reference baselines.

2.7 Short reflections

- **Q2.5 (one shot vs recursive).** The model is one-shot (direct). Errors do not compound, but early steps are less sharp than persistence because the model is optimizing all 168 steps together. Recursive rollout would make the first few steps easier but would accumulate errors over a heavy-tailed week.
- **Q2.6 (window normalisation).** Removing each window’s level destroys the absolute level, which here carries information (very high levels tend to decay within days). Global scaling keeps it. Anchoring on the last value did not help once position marks were present.

- **Q2.7 (past vs known-horizon covariates).** The distinction changes where a covariate enters. Here the same ten variables were worthless as past-only encoder inputs and cut RMSE by 24% when passed through the decoder marks.
- **Q2.8 (no timestamps).** Calendar effects (weekday, holidays, heating season start) are lost. The 24-step period was recovered from the spectrum and autocorrelation and encoded from the integer index. The yearly encoding (8 766 steps) is an assumption with weak support in the data. This is only trustworthy because the sampling is uniform with no gaps, as the task guarantees.

Generative AI use

I used Claude (Anthropic) and OpenAI Codex as assistants for this assignment.

Prompts.

- I asked Claude to read the PA1 handout and notebook, complete the required implementation cells, run the notebook on the full preset, and draft the Task 1 responses from the numbered outputs.
- I asked Claude to build a small Autoformer solution for the leaderboard task, including preprocessing, chronological validation, optional-data ablation, multi-seed runs, plots, and the final 168-value submission string.
- I asked Codex to review Claude’s work against the assignment requirements, check for over-engineering or non-student-like writing, verify the leaderboard submission files, and make the final report submission-ready.

What the assistant produced.

- Task 1: the three notebook implementations, the full-preset run, and first drafts of Responses 1A–4 from the exported Outputs.
- Task 2: the `pa1q2` package (data handling, from-scratch Autoformer, training with resumable early stopping, baselines, aggregation, figures, submission writer), the experiment plan and runs, and a first draft of this section.
- Final review: checks for missing fields, exact prediction count, declared parameter and epoch values, and consistency between the report and saved outputs.

My edits and checks.

- I checked the notebook outputs, kept the full-preset figures and tables, and edited the responses to focus on the required comparisons instead of only describing the code.
- For Task 2, I used my own validation split to choose the final model, kept the optional-data ablation, and did not use the public leaderboard as a validation signal.
- I verified that `submission/predictions.txt` contains exactly 168 values in chronological order and that the declared parameter count and epoch count match the saved final run.

References

- Wu, Xu, Wang, Long (2021). Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. NeurIPS. Reference code: <https://github.com/thuml/Autoformer>.
- Vaswani et al. (2017). Attention Is All You Need.
- Zeng et al. (2023). Are Transformers Effective for Time Series Forecasting?
- Hyndman and Athanasopoulos. Forecasting: Principles and Practice.